

PlantHealthMonitor

Final Project Report

Casey Stijlaart

Table of Contents

1. Introduction	1
1.1. Project Context	1
1.2. Project Goal	1
1.3. Research Questions	2
2. Background Research	3
2.1. Related Work	3
2.2. Technical Background	3
2.2.1. Edge AI and TinyML	3
2.2.2. Model Selection	4
2.2.3. Sensor Data and Preprocessing	4
2.2.4. Datasets	4
3. System Design	5
3.1. Requirements	5
3.1.1. User Requirements	5
3.1.2. Key Functional Requirements	5
3.1.3. Key Non-Functional Requirements	5
3.2. Architecture	6
3.3. Use Cases	7
3.4. Component Overview	8
4. Proof of Concept Iterations	9
4.1. PoC-01: Initial ML Approach	9
4.2. PoC-02: Dataset Redesign and Model Revision	9
4.3. PoC-03: Embedded Deployment and Validation	10
5. Implementation	11
5.1. Firmware (ESP32)	11
5.2. Mobile Application (Flutter)	11
5.3. Technologies Used	12
6. Testing and Validation	13
6.1. Model Validation	13
6.2. Functional Testing	13
6.3. Embedded Performance	14
7. Results	15
8. Challenges Encountered	16
8.1. Dataset Design	16
8.2. Embedded Constraints	16
8.3. Justifying ML Over Rule-Based Logic	16
8.4. Bridging the Training-to-Deployment Gap	17
8.5. WiFi Dependency for Configuration	17

8.6. Notification Reliability on iOS	17
9. Societal and Ethical Considerations.	18
9.1. Privacy.	18
9.2. Sustainability.	18
9.3. Psychological Impact	18
9.4. AI Augmentation.	18
10. Lessons Learned.	19
10.1. ML is only justified by the data.	19
10.2. Iterative prototyping surfaces problems early	19
10.3. Embedded development requires end-to-end thinking	19
10.4. User experience matters as much as technical accuracy	19
11. Future Improvements.	20
12. Conclusion	21

1. Introduction

1.1. Project Context

This project was developed as part of the AI for Society minor at Fontys University of Applied Sciences, Eindhoven. The project addresses a common problem: many people struggle with consistent, informed plant care. Generic advice is often unhelpful, and most existing solutions either require expensive cloud infrastructure or provide no intelligent analysis at all.

PlantHealthMonitor (PHM) applies AI techniques in an embedded, privacy-respecting setting to bridge that gap, providing real-time insights and care recommendations directly on the device, without depending on external services for inference.

1.2. Project Goal

The primary goal was to gain practical experience with integrating AI into a real-world embedded system by designing and implementing a smart desk plant monitoring system that provides real-time insights and actionable care recommendations for indoor plants.

The project was also an opportunity to engage with the full development cycle: background research, prototyping, iterative redesign, system design, implementation, and critical reflection on societal impact.

1.3. Research Questions

The central research question guiding the project was:

How can a smart desk plant monitoring system use AI to provide accurate, user-friendly care recommendations while addressing ethical, legal, and societal considerations?

This was broken down into three categories of sub-questions:

Technical / AI

- Which AI model types are suitable for classifying plant health on an embedded microcontroller?
- Do existing trained models exist, or does a custom dataset need to be built?
- Which sensors are appropriate for monitoring indoor plant conditions?
- How can a lightweight ML model be deployed and validated on an ESP32?

User Interaction

- How should care recommendations be presented to be actionable and non-alarming?
- What usability features make the system accessible to non-technical users?

Ethical and Legal

- What are the privacy implications of collecting continuous sensor data?
- What are the ethical responsibilities around automating plant care decisions?

2. Background Research

2.1. Related Work

A review of existing plant monitoring systems identified three categories:

Category	Cost	Target Use	AI Capability
IoT-Based (non-AI)	Low	Hobbyist / personal	Threshold automation only
AI/ML-Based	Low-medium	Research / experimental	Advanced classification, limited deployment
Commercial Solutions	High	Large-scale agriculture	Advanced sensors, proprietary models

The comparison identified a clear gap: there are no affordable, small-scale AI/ML-based plant monitoring systems designed specifically for personal indoor plant care. Existing research-grade systems are not deployable on consumer hardware, and commercial systems target agriculture rather than individual users. PHM was designed to fill this gap.

2.2. Technical Background

2.2.1. Edge AI and TinyML

Edge AI refers to running machine learning inference directly on embedded hardware rather than in the cloud. For plant monitoring this offers three key advantages: resilience (no internet dependency), privacy (sensor data stays on device), and latency (instant response without a network round-trip).

The ESP32 microcontroller, while constrained in memory and compute, is capable of executing lightweight ML workloads. Two established frameworks enable this: TensorFlow Lite Micro (TFLM) and Edge Impulse. Both support quantized models that fit within the ESP32's available SRAM and flash.

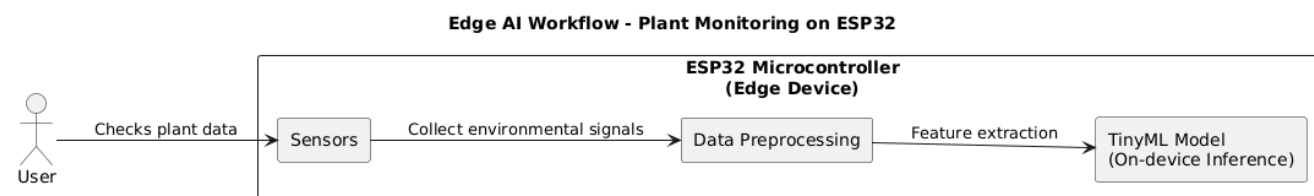


Figure 1. Edge AI Workflow — Plant Monitoring on ESP32

2.2.2. Model Selection

Research into models suitable for embedded systems identified the following candidates:

- **Decision trees** — fast, interpretable, low memory.
- **Linear regression** — simple, efficient, suitable for regression tasks.
- **Small neural networks (MLP)** — more expressive, trainable with standard tools, exportable to TFLM.
- **Threshold-based anomaly detection** — deterministic, zero inference cost.

For PHM, a small MLP was selected for both the health risk classifier and the predictive irrigation regression model, due to its ability to learn non-linear relationships between sensor features while remaining small enough for on-device deployment.

2.2.3. Sensor Data and Preprocessing

The system monitors four environmental factors: soil moisture, temperature, humidity, and light intensity. On the ESP32, preprocessing steps include normalisation, noise filtering, and feature extraction from a sliding window of historical readings. This temporal context is essential for time-to-action predictions (e.g., "water in 4 hours") that could not be derived from a single instantaneous reading.

2.2.4. Datasets

No existing public dataset adequately matched the project's use case. Available Kaggle datasets for plant health were either image-based (not applicable to sensor data) or lacked the multivariate, time-series structure needed for the predictive model. This led to the decision to generate a custom synthetic dataset — a decision that became a significant learning point during prototyping (see [Proof of Concept Iterations](#)).

3. System Design

3.1. Requirements

Requirements were derived from the project goal and structured using MoSCoW prioritization.

3.1.1. User Requirements

ID	Requirement
UR_001	The user can add a new plant with care requirements.
UR_002	The user can monitor the health status of a plant.
UR_003	The user can view current and historical sensor data.
UR_004	The user can update plant care settings.
UR_005	The user receives a notification when an abnormal condition is detected.

3.1.2. Key Functional Requirements

- The system collects soil moisture, temperature, humidity, and light readings from sensors connected to an ESP32.
- The ESP32 preprocesses sensor data and runs on-device ML inference to classify plant health.
- Classification results and care recommendations are transmitted to a cloud database (Supabase).
- A cross-platform Flutter app subscribes to live data and displays health status, sensor readings, and recommendations.
- The app sends push notifications when risk class changes to moderate or high.
- The user can configure per-plant care preferences (low / mid / high per sensor).

3.1.3. Key Non-Functional Requirements

- The ML model must fit within the ESP32's available flash and SRAM.
- The system must function without internet access for on-device inference.
- Sensor data must not include personally identifiable information.
- The system must support multiple plants simultaneously.

3.2. Architecture

The system is organised into three layers:

Layer	Component	Responsibility
Edge	ESP32 + sensor array	Sensor acquisition, preprocessing, ML inference, cloud sync every 3 hours.
Cloud	Supabase (PostgreSQL + Realtime)	Persistent storage of readings and settings; realtime change events to connected clients.
App	Flutter (iOS / Windows)	Live dashboard, charts, preferences editor, push notifications.

The edge layer operates independently. If the network is unavailable, the ESP32 continues collecting and classifying data locally, buffering readings in a CSV history on its filesystem (LittleFS). When connectivity is restored, data is synced to the cloud.

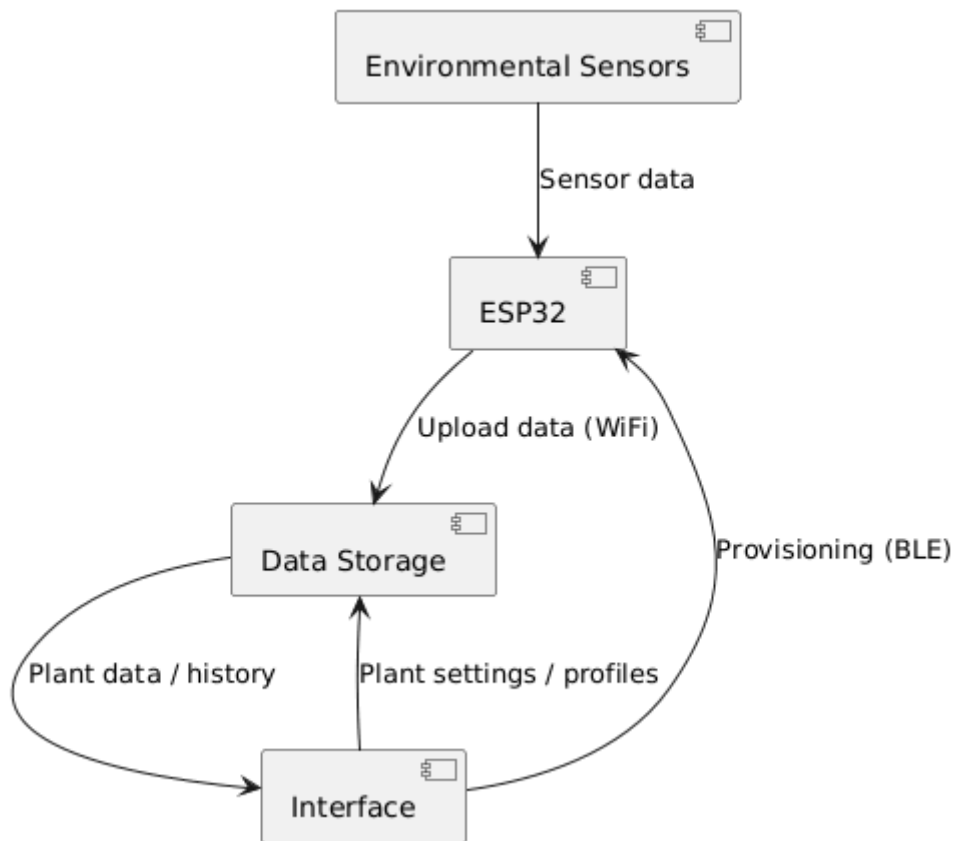


Figure 2. System Architecture — Hardware and Data Flow

3.3. Use Cases

Five use cases define the system's functional scope:

- **UC01 — Add Plant:** The user adds a plant with a label and sets initial care preferences. These are saved to the cloud and picked up by the device on its next sync.
- **UC02 — Monitor Plant:** The ESP32 continuously reads sensors, runs inference, and uploads results. The app reflects the latest status in real time via a Supabase Realtime subscription.
- **UC03 — View Plant Data:** The user opens the data view to see sensor history charts for the last week (up to 56 readings at 3-hour intervals).
- **UC04 — Update Settings:** The user edits care preferences in the app. A version counter prevents stale overwrites when settings are synced across devices.
- **UC05 — Abnormal Condition Alert:** When risk class changes to moderate (1) or high (2), the app generates a push notification. Duplicate notifications are suppressed.

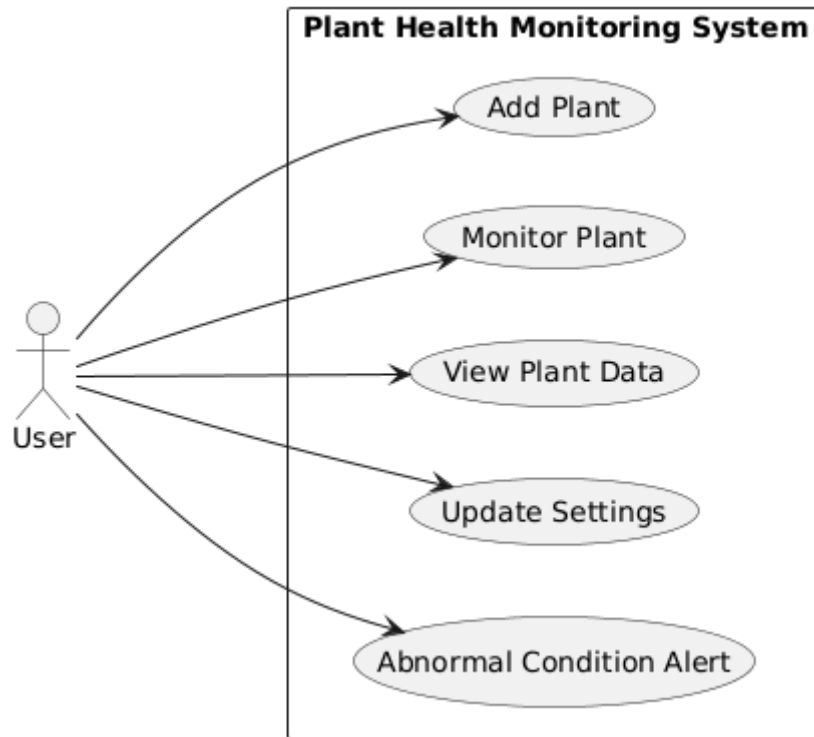


Figure 3. Use Case Diagram — Plant Health Monitoring System

3.4. Component Overview

Component	Responsibility
Edge Node (ESP32)	Reads sensors, preprocesses data (normalisation, feature extraction, temporal windowing), runs ML inference via the pluggable <code>MLayer</code> , builds recommendations via <code>RecommendationEngine</code> , and POSTs to Supabase.
Sensor Array	Capacitive soil moisture sensor (GPIO 34), DHT22 temperature + humidity sensor (GPIO 4), LDR light sensor (GPIO 35).
MLayer	Pluggable abstraction supporting rule-based classification, TFLM MLP classifier, and Edge Impulse models.
RecommendationEngine	Combines raw sensor values, preference thresholds, and ML risk class to produce boolean action flags and calls <code>PredictiveIrrigationModel</code> for the time estimate.
PredictiveIrrigationModel	Regression MLP (trained offline, exported as C++ header weights) that predicts minutes until watering is needed.
Cloud (Supabase)	Two tables: <code>plant_readings</code> (sensor data, risk class, recommendations) and <code>plant_settings</code> (per-plant preferences with version control).
Flutter App	Realtime dashboard, plant selector, sensor history charts, preferences editor, notification scheduler.

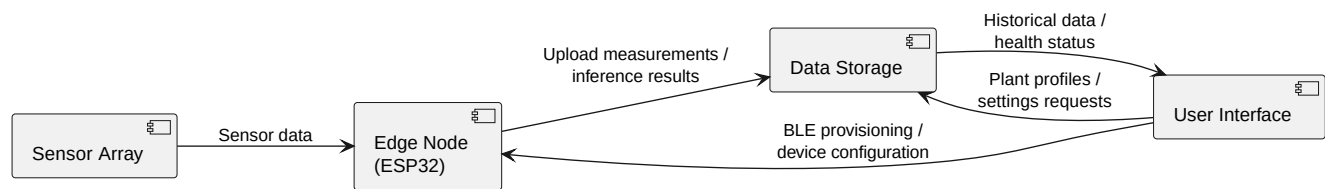


Figure 4. Component Diagram — System Components and Interactions

4. Proof of Concept Iterations

The system was developed through three proof-of-concept iterations. Each uncovered problems that led to a redesign before the next iteration.

4.1. PoC-01: Initial ML Approach

The first prototype trained logistic regression, MLP, and random forest classifiers on a publicly available plant health dataset.

Key findings:

- The models achieved high accuracy, but analysis revealed they were replicating a single threshold on soil moisture. Temperature and humidity contributed almost nothing to predictions.
- The dataset did not represent real-world plant health dynamics — it lacked temporal information and meaningful multi-sensor interactions.
- The models could have been replaced by a single `if soil < 35%` rule with equivalent accuracy.

Conclusion: The dataset was unsuitable and the ML approach was not justified. A redesign was needed.

4.2. PoC-02: Dataset Redesign and Model Revision

PoC-02 focused entirely on building a dataset that would force the model to learn genuine multi-factor relationships.

Changes made:

- The synthetic data generation was redesigned to introduce time-dependent environmental variables, feature interactions (e.g., high temperature amplifying the impact of low humidity), and realistic stress scenarios.
- Pre-engineered features that encoded domain logic (effectively bypassing the model) were removed.
- Multiple model architectures were re-evaluated on the new dataset.

Results:

- Feature importance analysis confirmed that all four sensor inputs contributed meaningfully to predictions.
- The MLP generalised to unseen conditions rather than memorising a threshold.
- The revised dataset demonstrated that a well-designed training set is as important as model architecture when justifying ML over simpler alternatives.

Conclusion: The redesigned dataset and model were ready for embedded deployment.

4.3. PoC-03: Embedded Deployment and Validation

PoC-03 moved the validated model onto the ESP32 and added the full system stack.

Changes and additions:

- The TFLM model was deployed via the `MLayer` abstraction, allowing the rule-based fallback to remain available for comparison.
- A CSV logger was added using LittleFS to persist a week of sensor history on the device.
- A lightweight HTTP server was added to the ESP32 for local inspection of live data and history.
- NTP time synchronisation was implemented to ensure accurate timestamps in the cloud.
- User personalisation was introduced: per-plant preference profiles stored in Supabase and read by the `RecommendationEngine`.

PoC-03 — ESP32 Local Web Interface for Plant Settings and Log Files

image::.../proof-of-concept/images/plant1-server.png[

Validation:

On-device TFLM inference was compared against the same model running in Python (scikit-learn / numpy). Predictions were identical across all test vectors, confirming that the C++ weight export and inference pipeline were correct.

Conclusion: The embedded system performed as expected. The validation confirmed model consistency between the training environment and the deployed device.

5. Implementation

5.1. Firmware (ESP32)

The production firmware (`src/prototype-non-local/`) is built with PlatformIO. The main monitoring loop runs every 3 hours:

1. Sensor readings are acquired from the three sensor types.
2. Readings are normalised against per-plant preference thresholds.
3. A feature vector is constructed, including temporal features derived from the circular history buffer stored in CSV on LittleFS.
4. The `MLLayer` runs the TFLM classifier and returns a risk class (0/1/2) with per-class confidence scores.
5. The `PredictiveIrrigationModel` runs the regression MLP and returns a time estimate in minutes.
6. The `RecommendationEngine` combines sensor values, preference thresholds, ML risk class, and the time estimate to produce a `Recommendation` struct (boolean action flags + summary string).
7. The result is serialised to JSON and uploaded to Supabase via HTTPS.

WiFi credentials and plant identity are configured at build time via `platformio.ini` build flags. OTA firmware updates are supported over WiFi for field updates.

5.2. Mobile Application (Flutter)

The Flutter app (`src/phm_app/`) targets iOS and Windows. Key implementation details:

- A Supabase Realtime subscription delivers `plant_readings` inserts to the app in real time without polling.
- The dashboard renders a status card per plant, updating reactively when new data arrives.
- Sensor tiles show the current reading and highlight whether it falls within the plant's preference band.
- The preferences editor reads and writes to the `plant_settings` table with an incrementing version field to handle concurrent edits.
- Push notifications are scheduled locally using `flutter_local_notifications` with timezone-aware scheduling for the daily summary feature.
- Notification deduplication prevents repeated alerts for the same persisting condition.

5.3. Technologies Used

Technology	Role
ESP32 microcontroller	Edge processing, sensor acquisition, on-device ML inference.
PlatformIO	Firmware build system and dependency management.
TensorFlow Lite Micro (TFLM)	On-device neural network inference.
LittleFS	Filesystem for CSV history on ESP32 flash.
Supabase	Managed PostgreSQL + Realtime backend (cloud layer).
Flutter	Cross-platform mobile and desktop application.
fl_chart	Sensor history charts in the Flutter app.
flutter_local_notifications	Push notifications and daily summary scheduling.
Python / scikit-learn	Offline model training and validation.

6. Testing and Validation

6.1. Model Validation

The most critical validation step was confirming that on-device TFLM inference matched the offline Python model exactly. This was done by:

1. Running both models against the same set of test feature vectors.
2. Comparing the risk class output and confidence scores for each vector.
3. Confirming identical results across all test cases.

This confirmed that the C++ weight export, feature normalisation, and inference pipeline introduced no numerical errors.

```
PS C:\Users\casey\Desktop\Fontys\sem-7-AI\personal-project\src\pof-02\python> & C:/Users/casey/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/casey/Desktop/Fontys/sem-7-AI/personal-project/src/pof-02/python/r1_data-valid.py
Input rows: 22
Wrote predictions to: C:\Users\casey\Desktop\Fontys\sem-7-AI\personal-project\src\pof-02\python\data\output\log_2026-04-18_model_predictions.csv

Model feature order:
['soil_now', 'moisture_mean', 'moisture_delta', 'moisture_slope', 'dry_duration', 'temp_now', 'humidity_now', 'light_now', 'hour_sin', 'hour_cos', 'dow_sin', 'dow_cos']

Predicted class distribution:
model_risk_class
0      1.0
Name: proportion, dtype: float64

Risk class agreement: 100.00%

Confusion matrix:
model_risk_class    0
logged_risk_class
0                  22

Mismatch rows: 0
```

Figure 5. Model Validation Output — 100% Risk Class Agreement Between TFLM and Python

6.2. Functional Testing

Each use case was tested manually against the running system:

Use Case	Test	Result
UC01	Add a plant in the app; verify it appears on the device's next sync.	Pass
UC02	Observe live sensor readings updating in the app within seconds of a device reading.	Pass
UC03	Navigate to the data view and verify charts populate with historical readings.	Pass
UC04	Change a preference in the app; verify the device picks it up on the next cycle.	Pass
UC05	Trigger an abnormal reading; verify a push notification is received.	Pass

6.3. Embedded Performance

The TFLM model and full firmware stack were verified to fit within the ESP32's memory constraints. Inference time per cycle was negligible relative to the 3-hour interval between readings. Power consumption was not formally measured but the 3-hour sleep interval between active cycles makes the system suitable for low-power operation with minor modifications (light sleep mode).

7. Results

The completed system meets all must-have requirements and the majority of should-have requirements defined at the start of the project.

- A fully functional embedded plant monitoring device was built and deployed.
- On-device ML inference (risk classification and predictive irrigation) runs correctly on the ESP32 without cloud dependency.
- A cross-platform Flutter application displays live sensor data, health status, recommendations, and historical charts.
- Push notifications alert users to plant care needs in real time.
- The system supports multiple plants simultaneously with per-plant preference profiles.
- The full development cycle — from background research and dataset design through to embedded deployment and cross-platform app — was completed within the semester.

The iterative prototyping process was particularly valuable. The failures of PoC-01 and PoC-02 led directly to a better understanding of what makes ML justifiable over simpler rule-based approaches, and resulted in a more robust final implementation.

8. Challenges Encountered

8.1. Dataset Design

The most significant technical challenge was designing a dataset that genuinely required ML rather than replicating a threshold rule. The first iteration wasted considerable time before feature importance analysis revealed the problem. The lesson — that a poorly designed dataset produces a dishonest model regardless of architecture — fundamentally shaped the rest of the project.

8.2. Embedded Constraints

Deploying a neural network on the ESP32 required careful attention to quantisation, weight export format, and memory layout. The TFLM inference pipeline had to be implemented in C++ with fixed-size arrays, and the normalisation parameters had to be exported alongside the weights to ensure inference matched training exactly.

8.3. Justifying ML Over Rule-Based Logic

One of the less obvious but more important challenges was justifying the use of machine learning at all. Early in the project it became clear that a naive model trained on a poorly designed dataset would simply learn to replicate a single threshold rule — something a five-line `if` statement could do without any ML overhead.

Proving that ML added genuine value required designing the dataset such that no single sensor dominated predictions and that the model was forced to learn multi-factor, non-linear relationships. This meant multiple rounds of feature importance analysis, dataset regeneration, and re-evaluation before the model's behaviour could be considered honest. The pluggable `MLayer` architecture also meant a rule-based classifier remained available as a direct comparison baseline throughout, which helped make this justification concrete.

8.4. Bridging the Training-to-Deployment Gap

Training a model in Python with scikit-learn and deploying it as a TFLM C++ weight export are two very different things. Several challenges arose at this boundary:

- Normalisation parameters (mean and standard deviation per feature) had to be exported and hard-coded into `ModelExport.hpp` alongside the weights. Any mismatch between training-time and inference-time scaling causes silent errors — the model produces wrong predictions without raising any errors.
- The regression model's training target used a `log1p` transform to handle skewed minute values. The inverse transform (`expm1`) had to be applied at inference time in C++, which was easy to forget and hard to debug without the Python reference implementation alongside.
- TFLM requires fixed-size arrays and does not support dynamic allocation. The model architecture had to be fixed before export, and any subsequent change to the number of features or hidden units required regenerating the header and rebuilding the firmware.

The validation step in PoC-03 — running both models side-by-side on the same test vectors — was the only reliable way to catch these issues, and doing it systematically is what gave confidence in the final deployment.

8.5. WiFi Dependency for Configuration

The current setup requires WiFi credentials to be compiled into the firmware, which is impractical for end users. This was identified as a known limitation early and has been documented as a future improvement (WiFi provisioning / BLE configuration).

8.6. Notification Reliability on iOS

iOS applies strict restrictions on background processes and notification permissions that made the local notification system more complex to implement reliably than on Android. The 7-day certificate expiry imposed by free Apple ID sideloading also means iOS users face a recurring friction point, although this is only relevant for non-EU users due to Apple's policy changes.

9. Societal and Ethical Considerations

9.1. Privacy

The system processes environmental sensor data only. No personal information is collected. The Supabase backend stores readings identified by a `plant_label` string chosen by the user, with no link to personal identity. The on-device ML inference means raw sensor data never needs to leave the device for classification.

Indirect profiling risks (e.g., inferring household occupancy patterns from lighting data) were acknowledged in the impact assessment. Mitigation recommendations include strict data retention policies and user control over what is transmitted to the cloud.

9.2. Sustainability

Running ML inference on-device rather than in the cloud reduces energy consumption associated with cloud compute. However, the embedded hardware itself has an environmental cost. The design mitigates this through modularity (sensors can be replaced independently), and future versions should select components rated for longevity and design for repairability.

9.3. Psychological Impact

The system was designed to offer suggestions rather than commands. Non-alarming language and transparent explanations of how recommendations are derived are essential to prevent over-reliance and user anxiety. The psychological impact assessment identified a risk that users might reduce their own observational engagement with their plants if they defer entirely to the system. The recommendation engine was therefore designed to provide context (sensor values and preference bands) alongside each suggestion, encouraging informed decision-making rather than passive compliance.

9.4. AI Augmentation

AI tools were used throughout the project in a supporting role:

- **ChatGPT Codex** — code review, debugging, and HTML/CSS snippets for the PoC-03 web interface.
- **ChatGPT** — brainstorming, documentation review, and researching non-local access solutions.
- **Schematik.io** — hardware schematic and circuit design.
- **Claude** — brainstorming, verification, and image generation (app startup screen and logo).
- **Claude Code** — Flutter front-end development, debugging, and code review.

All AI-generated content was reviewed, verified, and adapted before use. The tools accelerated iteration but did not replace understanding — particularly important when debugging embedded inference, where AI suggestions had to be validated against hardware behaviour.

10. Lessons Learned

10.1. ML is only justified by the data

The most important lesson of the project came from PoC-01: a model is only as good as its training data. A model that replicates a threshold rule provides no value over the rule itself. Designing data that requires genuine multi-factor learning is a prerequisite for honest ML, not an afterthought.

10.2. Iterative prototyping surfaces problems early

Each PoC was deliberately scoped to answer a specific question before committing to the next layer of complexity. This kept the overall risk low and ensured that the final implementation was built on validated decisions rather than assumptions.

10.3. Embedded development requires end-to-end thinking

Choices made at training time (normalisation parameters, output range, quantisation) have direct consequences at inference time on the device. The validation step comparing Python and TFLM output was essential and caught subtle mismatches in feature scaling before they reached production.

10.4. User experience matters as much as technical accuracy

A technically correct recommendation that is presented in alarming or confusing language causes more harm than it prevents. The psychological impact assessment reinforced the importance of framing AI output as a suggestion that supports user decision-making, not as an authoritative command.

11. Future Improvements

Several improvements were identified during the project but were out of scope for this semester. These are documented in the manual and summarised here:

- **WiFi provisioning** — allow network credentials and plant identity to be configured from the app on first boot, removing the need for firmware recompilation.
- **BLE configuration and sync** — use Bluetooth Low Energy as an alternative channel for device setup and offline data access.
- **Built-in plant species library** — ship pre-defined care profiles for common species so users do not need to configure preferences manually.
- **Additional sensors** — pH, electrical conductivity, CO₂, and water level sensors would extend the system's applicability.
- **Automated watering** — connect a pump and relay to act on irrigation predictions autonomously.
- **Continuous ML retraining** — collect anonymised data from deployed devices to improve models over time via OTA weight updates.
- **Multi-user support** — authentication and shared plant ownership for households with multiple users.

12. Conclusion

PlantHealthMonitor demonstrates that meaningful AI-driven plant care is achievable on affordable embedded hardware without cloud dependency for inference. The project delivered a working end-to-end system: sensors on an ESP32 running a trained neural network, syncing to a cloud backend, and surfacing results in a cross-platform mobile app with real-time notifications.

Beyond the technical deliverable, the project produced genuine learning outcomes around the responsible application of ML — particularly the importance of dataset design in justifying ML over simpler alternatives, and the ethical responsibility of presenting AI-generated suggestions in a way that preserves user autonomy and trust.

The iterative prototyping approach was essential. The failures of the first two PoC iterations were not setbacks; they were the mechanism by which the final system's foundations were validated. This reflects a broader principle: in applied AI development, honest evaluation of what is not working is as valuable as celebrating what is.